PYTHON PROGRAMMING (ABL) REPORT

Name:	Md Zaheeruddin	**Roll No:**	26000179
Branch & Sec:	CSE-3	**Course:**	Python Programming (ABL)

Case 27: Advanced Numeric Pattern Generator

Problem Statement: Produce a hollow diamond or symmetric numerical pattern based on user-specified dimensions, with validation for invalid dimensions and carefully aligned output.

Python Source Code:

```
def generate_numeric_diamond(n):
    if n <= 0 or n % 2 == 0:
        print("Please enter a positive odd integer.")
        return
    print(f"--- Symmetric Hollow Diamond Pattern (Size: {n}) ---\n")
    spaces = n // 2
    for i in range(1, n + 1, 2):
        row_str = " " * spaces
        if i == 1:
            row_str += str(1)
        else:
            row_str += str(i) + " " * (2 * (i - 2) + 1) + str(i)
        print(row_str)
        spaces -= 1
    spaces = 1
    for i in range(n - 2, 0, -2):
        row_str = " " * spaces
        if i == 1:
            row_str += str(1)
        else:
            row_str += str(i) + " " * (2 * (i - 2) + 1) + str(i)
        print(row_str)
        spaces += 1
generate_numeric_diamond(7)
```

Execution Output:

```
--- Symmetric Hollow Diamond Pattern (Size: 7) ---
   1
  3 3
 5   5
7     7
 5   5
  3 3
   1
```

Case 13: ATM Transaction Sequence Simulation

Problem Statement: Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Python Source Code:

```
def simulate_atm(opening_balance, transaction_history):
    print("--- ATM Transaction Sequence Report ---\n")
    balance = opening_balance
    daily_withdrawal_limit = 20000
    total_withdrawn_today = 0
    successful_txns = 0
    rejected_txns = 0
    for txn in transaction_history:
        txn_type, amount = txn
        if amount <= 0:
            print(f"REJECTED: Invalid amount {amount}")
            rejected_txns += 1
            continue
        if txn_type == "Deposit":
            balance += amount
            successful_txns += 1
            print(f"SUCCESS : Deposited Rs.{amount}. Current Balance: Rs.{balance}")
        elif txn_type == "Withdrawal":
            if amount > balance:
                print(f"REJECTED: Insufficient Funds for Withdrawal of Rs.{amount} (Balance:
 Rs.{balance})")
                rejected_txns += 1
            elif total_withdrawn_today + amount > daily_withdrawal_limit:
                print(f"REJECTED: Exceeded Daily Withdrawal Limit for Rs.{amount}")
                rejected_txns += 1
            else:
                balance -= amount
                total_withdrawn_today += amount
                successful_txns += 1
                print(f"SUCCESS : Withdrew Rs.{amount}. Current Balance: Rs.{balance}")
    print("\n--- Summary ---")
    print(f"Final Balance       : Rs.{balance}")
    print(f"Successful Actions  : {successful_txns}")
    print(f"Rejected Actions    : {rejected_txns}")
transactions = [("Deposit", 5000), ("Withdrawal", 12000), ("Withdrawal", 10000), ("Deposit",
 -500), ("Withdrawal", 2000)]
simulate_atm(15000, transactions)
```

Execution Output:

```
--- ATM Transaction Sequence Report ---

SUCCESS : Deposited Rs.5000. Current Balance: Rs.20000
SUCCESS : Withdrew Rs.12000. Current Balance: Rs.8000
REJECTED: Insufficient Funds for Withdrawal of Rs.10000 (Balance: Rs.8000)
REJECTED: Invalid amount -500
SUCCESS : Withdrew Rs.2000. Current Balance: Rs.6000

--- Summary ---
Final Balance       : Rs.6000
Successful Actions  : 3
Rejected Actions    : 2
```

Case 2: Progressive Electricity Billing

Problem Statement: Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

Python Source Code:

```
def calculate_electricity_bill(units):
    fixed_charge = 150
    bill_amount = fixed_charge
    if units <= 100:
        bill_amount += units * 4.5
    elif units <= 300:
        bill_amount += (100 * 4.5) + (units - 100) * 6.0
    else:
        bill_amount += (100 * 4.5) + (200 * 6.0) + (units - 300) * 8.5
    if bill_amount > 2000:
        bill_amount += bill_amount * 0.05
    return round(bill_amount, 2)

monthly_consumption = [80, 250, 450]
print("--- Multi-Month Consumption Audit ---")
previous_bill = 0
for month_idx in range(len(monthly_consumption)):
    units = monthly_consumption[month_idx]
    current_bill = calculate_electricity_bill(units)
    alert_flag = ""
    if month_idx > 0 and current_bill > previous_bill * 1.5:
        alert_flag = "[UNUSUAL SPIKE DETECTED]"
    print(f"Month {month_idx + 1}: {units} Units -> Cost: Rs.{current_bill} {alert_flag}")
    previous_bill = current_bill
```

Execution Output:

```
--- Multi-Month Consumption Audit ---
Month 1: 80 Units -> Cost: Rs.510.0
Month 2: 250 Units -> Cost: Rs.1200.0 [UNUSUAL SPIKE DETECTED]
Month 3: 450 Units -> Cost: Rs.3228.75 [UNUSUAL SPIKE DETECTED]
```

Case 29: Password Policy Validation Engine

Problem Statement: Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

Python Source Code:

```
def validate_password(password):
    failures = []
    if len(password) < 8:
        failures.append("Must be at least 8 characters long.")
    has_upper = has_lower = has_digit = False
    for char in password:
        if 'A' <= char <= 'Z': has_upper = True
        elif 'a' <= char <= 'z': has_lower = True
        elif '0' <= char <= '9': has_digit = True
    if not has_upper: failures.append("Missing an uppercase letter (A-Z).")
    if not has_lower: failures.append("Missing a lowercase letter (a-z).")
    if not has_digit: failures.append("Missing a numeric digit (0-9).")
    if "123" in password or "password" in password.lower():
        failures.append("Contains a prohibited weak pattern ('123' or 'password').")
    print(f"Evaluating: '{password}'")
    if len(failures) == 0: print("Verdict: STRONG PASSWORD\n")
    else:
        print("Verdict: WEAK PASSWORD")
        for fail in failures: print(f" -> Failed Rule: {fail}")
        print()
validate_password("weak")
validate_password("ValidP@ss2026")
```

Execution Output:

```
Evaluating: 'weak'
Verdict: WEAK PASSWORD
 -> Failed Rule: Must be at least 8 characters long.
 -> Failed Rule: Missing an uppercase letter (A-Z).
 -> Failed Rule: Missing a numeric digit (0-9).

Evaluating: 'ValidP@ss2026'
Verdict: STRONG PASSWORD
```

Case 35: Text Compression Using Character Runs

Problem Statement: Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

Python Source Code:

```
def run_length_compress(input_str):
    if not input_str: return ""
    compressed_list = []
    count = 1
    for i in range(1, len(input_str)):
        if input_str[i] == input_str[i - 1]: count += 1
        else:
            compressed_list.append(input_str[i - 1] + str(count))
            count = 1
    compressed_list.append(input_str[-1] + str(count))
    compressed_string = "".join(compressed_list)
    print(f"Original Text   : {input_str} (Length: {len(input_str)})")
    print(f"Compressed Text : {compressed_string} (Length: {len(compressed_string)})")
    if len(compressed_string) >= len(input_str):
        print("Decision        : Compression rejected (No data footprints optimization optim
ization value).\n")
    else:
        print("Decision        : Compression accepted (Efficient footprint reduction).\n")
run_length_compress("WWWWWWWWWWWWBWWWWWWWWWWWWBBB")
run_length_compress("abcdefg")
```

Execution Output:

```
Original Text   : WWWWWWWWWWWWBWWWWWWWWWWWWBBB (Length: 28)
Compressed Text : W12B1W12B3 (Length: 10)
Decision        : Compression accepted (Efficient footprint reduction).

Original Text   : abcdefg (Length: 7)
Compressed Text : a1b1c1d1e1f1g1 (Length: 14)
Decision        : Compression rejected (No data footprints optimization optimization value).
```

Case 7: Retail Billing with Multiple Discount Rules

Problem Statement: Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

Python Source Code:

```
def generate_invoice(cart_items):
    print("================ INVOICE GENERATOR ================ ")
    subtotal = total_tax = 0.0
    for item in cart_items:
        name, qty, price, category = item
        item_cost = qty * price
        tax_rate = 0.18 if category == "Electronics" else 0.05
        item_tax = item_cost * tax_rate
        subtotal += item_cost
        total_tax += item_tax
    gross_total = subtotal + total_tax
    discount = 0.0
    if gross_total > 5000:
        discount = gross_total * 0.10
        if discount > 1000: discount = 1000.0
    final_payable = gross_total - discount
    print(f"Gross Subtotal : Rs.{subtotal:.2f}")
    print(f"Collected Tax  : Rs.{total_tax:.2f}")
    print(f"Applied Savings: -Rs.{discount:.2f}")
    print(f"Net Payable    : Rs.{final_payable:.2f}")
shopping_cart = [("Laptop Bag", 1, 1500.0, "Apparel"), ("USB Drive", 3, 600.0, "Electronics"
)]
generate_invoice(shopping_cart)
```

Execution Output:

```
================ INVOICE GENERATOR ================
Gross Subtotal : Rs.3300.00
Collected Tax  : Rs.399.00
Applied Savings: -Rs.0.00
Net Payable    : Rs.3699.00
```

Case 42: Student Marks Matrix Analysis

Problem Statement: Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Python Source Code:

```
def analyze_marks_matrix(students, subjects, matrix):
    num_students, num_subjects = len(students), len(subjects)
    subject_totals = [0] * num_subjects
    for i in range(num_students):
        student_sum = 0
        for j in range(num_subjects):
            score = matrix[i][j]
            student_sum += score
            subject_totals[j] += score
        print(f"Student: {students[i]:<8} | Total: {student_sum:<3} | Avg: {student_sum/num_
subjects:.2f}")
    print("\n--- Subject Metrics Trend ---")
    for j in range(num_subjects):
        subject_avg = subject_totals[j] / num_students
        print(f"Subject: {subjects[j]:<12} | System Mean: {subject_avg:.2f}")
student_roster = ["Amit", "Bhavna"]
subject_list = ["Maths", "Physics"]
marks_table = [[85, 90], [55, 58]]
analyze_marks_matrix(student_roster, subject_list, marks_table)
```

Execution Output:

```
Student: Amit     | Total: 175 | Avg: 87.50
Student: Bhavna   | Total: 113 | Avg: 56.50

--- Subject Metrics Trend ---
Subject: Maths        | System Mean: 70.00
Subject: Physics      | System Mean: 74.00
```

Case 31: String-Based Examination Answer Evaluation

Problem Statement: Compare answer strings against an answer key, calculate correct, incorrect, and unanswered responses, apply negative marking, and produce student-wise results.

Python Source Code:

```
def score_exam(answer_key, student_submissions):
    for record in student_submissions:
        name, response_str = record
        correct = incorrect = unanswered = 0
        for idx in range(len(answer_key)):
            if response_str[idx] == 'X': unanswered += 1
            elif response_str[idx] == answer_key[idx]: correct += 1
            else: incorrect += 1
        final_score = (correct * 4) + (incorrect * -1)
        print(f"{name} -> Summary: [{correct} Correct] | [{incorrect} Incorrect] | Score: {f
inal_score}")
key = "ABCDABCDAB"
submissions = [("John Doe", "ABCDBBCDAA")]\ score_exam(key, submissions)
```

Execution Output:

```
John Doe -> Summary: [8 Correct] | [2 Incorrect] | Score: 30
```

Case 49: Tuple-Based Product Catalogue Audit

Problem Statement: Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify items requiring replenishment.

Python Source Code:

```
def audit_product_catalogue(catalogue_tuples):
    total_val = 0.0
    for item in catalogue_tuples:
        prod_id, label, unit_price, stock_qty = item
        if unit_price < 0 or stock_qty < 0:
            print(f"DATA ERROR: Malformed entry for ID {prod_id}")
            continue
        total_val += (unit_price * stock_qty)
        if stock_qty <= 5:
            print(f"RESTOCK NOTICE: '{label}' is low! ({stock_qty} left)")
    print(f"Aggregate Asset Value: Rs.{total_val:.2f}")
warehouse_catalogue = [(101, "Keyboard", 4500.0, 12), (102, "Mouse", 1200.0, 3)]
audit_product_catalogue(warehouse_catalogue)
```

Execution Output:

```
RESTOCK NOTICE: 'Mouse' is low! (3 left)
Aggregate Asset Value: Rs.57600.00
```

Case 37: Nested Loop Calendar Generator

Problem Statement: Generate a formatted monthly calendar using the supplied starting weekday and number of days, correctly aligning weeks and handling valid month lengths.

Python Source Code:

```
def generate_monthly_calendar(start_day, days_in_month):
    print("Mon  Tue  Wed  Thu  Fri  Sat  Sun")
    print("     " * start_day, end="")
    col = start_day
    for day in range(1, days_in_month + 1):
        print(f"{day:<5}", end="")
        col += 1
        if col == 7:
            print()
            col = 0
    print()
generate_monthly_calendar(3, 31)
```

Execution Output:

```
Mon  Tue  Wed  Thu  Fri  Sat  Sun
               1    2    3    4
5    6    7    8    9    10   11
12   13   14   15   16   17   18
19   20   21   22   23   24   25
26   27   28   29   30   31
```